

ОГЛАВЛЕНИЕ

ВВЕДЕНИЕ.....	4
1 АНАЛИЗ ПРЕДМЕТНОЙ ОБЛАСТИ И ФОРМИРОВАНИЕ ТРЕБОВАНИЙ..	5
1.1 Анализ предметной области	5
1.2 Теоретическая база.....	6
1.3 Анализ конкурентных решений.....	7
1.4 Формирование требований к продукту.....	8
2 ПРОЕКТИРОВАНИЕ И ОБОСНОВАНИЕ ВЫБОРА ТЕХНОЛОГИЙ	10
2.1 Концепция продукта	10
2.2 Целевая аудитория	11
2.3 Обоснование выбора технологий	11
2.4 Анализ архитектуры и модульной структуры.....	12
2.5 Проектирование интерфейса.....	13
3 РЕАЛИЗАЦИЯ КЛИЕНТСКОЙ ЧАСТИ.....	15
3.1 Общая схема работы приложения	15
3.2 Реализация главной страницы	16
3.3 Реализация дашборда пользователя	18
3.4 Реализация страницы поездки	20
3.5 Реализация хранения данных.....	23
3.6 Работа с утилитами и безопасным выводом	25
4 ТЕСТИРОВАНИЕ, ДОКУМЕНТИРОВАНИЕ И ПУБЛИКАЦИЯ.....	27
4.1 Подход к тестированию.....	27
4.2 Документирование проекта.....	29
4.3 Публикация и запуск.....	29
ЗАКЛЮЧЕНИЕ	32
СПИСОК ИСПОЛЬЗУЕМЫХ ИСТОЧНИКОВ	33
ПРИЛОЖЕНИЕ А	34

ПРИЛОЖЕНИЕ Б.....**Ошибка! Закладка не определена.**

ВВЕДЕНИЕ

Современные веб-приложения для путешественников должны одновременно решать несколько прикладных задач: помогать планировать поездки, хранить важную информацию, обеспечивать доступ к погодным данным и поддерживать удобный пользовательский опыт. Чем меньше действий требуется для фиксации маршрута, дат, расходов и заметок, тем выше практическая ценность такого сервиса.

Актуальность темы обусловлена тем, что большинство существующих туристических платформ ориентированы на бронирование и коммерческую инфраструктуру. В учебных и бытовых сценариях пользователю часто нужен более легкий инструмент: он должен быть понятным, быстро открываться, не требовать сложной регистрации и позволять сохранить ключевые сведения о поездке в одном месте.

В рамках настоящей курсовой работы разработана клиентская часть веб-приложения Globe. Продукт предназначен для авторизации пользователя, создания списка поездок, управления подробной карточкой путешествия, отображения текущей погоды, ведения бюджета, чек-листа и текстовых заметок. Такое сочетание функций делает приложение удобным как для личного, так и для учебного использования.

Цель работы состоит в создании полнофункциональной клиентской части с модульной архитектурой и современным интерфейсом. Для достижения цели необходимо проанализировать предметную область, обосновать выбор стека технологий, спроектировать интерфейс, реализовать основные сценарии и проверить корректность работы приложения.

- проанализировать существующие решения в области планирования путешествий;
- определить функциональные и нефункциональные требования к приложению Globe;

- выбрать технологии для клиентской и вспомогательной серверной части;
- реализовать страницы и блоки интерфейса с использованием JavaScript и БЭМ;
- обеспечить хранение данных на стороне клиента и отображение погодных данных;
- провести тестирование и подготовить материал для публикации на GitHub Pages.

Практическая значимость работы заключается в том, что проект демонстрирует полный цикл фронтенд-разработки: от анализа требований и подготовки структуры проекта до реализации, тестирования и документирования. Полученное решение может быть расширено за счет подключения полноценного серверного хранилища, облачной синхронизации и совместного планирования поездок несколькими пользователями.

В дальнейшем текст отчета последовательно рассматривает предметную область, архитектурные решения, ход реализации и итоги проверки работоспособности. Такой порядок соответствует логике выполнения курсовой работы и позволяет связать проектные решения с конкретными функциями приложения.

АНАЛИЗ ПРЕДМЕТНОЙ ОБЛАСТИ И ФОРМИРОВАНИЕ ТРЕБОВАНИЙ

1.1 Анализ предметной области

Планирование путешествия включает несколько базовых этапов. Сначала пользователь выбирает направление и даты, затем оценивает бюджет, составляет список вещей, фиксирует полезные заметки и при необходимости отслеживает погодные условия. На практике эти действия нередко выполняются в разных сервисах и мессенджерах, из-за чего информация оказывается разрозненной и неудобной для последующего использования.

Для бытового и учебного сценария особенно ценна компактность интерфейса. Пользователь не хочет переходить между несколькими приложениями, регистрироваться в сложной системе или тратить время на длительное заполнение профиля. Поэтому в проекте Globe был выбран подход, при котором основные инструменты объединяются в одном интерфейсе, а доступ к ним начинается сразу после входа в систему.

Ключевыми характеристиками предметной области являются простота ввода данных, наглядность отображения информации и возможность быстро вернуться к ранее сохраненной поездке. Эти свойства определили структуру приложения: на главной странице пользователь знакомится с сервисом, на дашборде управляет поездками, а на детальной странице работает с погодой, бюджетом и заметками.

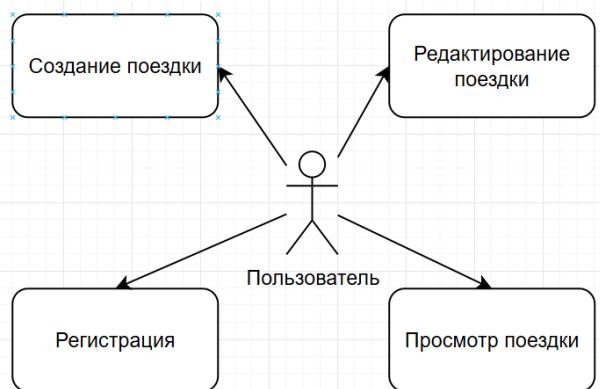


Рисунок 1 - Схема пользовательских сценариев приложения Globe

На схеме пользовательских сценариев видно, что основной поток работы строится вокруг трех экранов. Такое разделение уменьшает когнитивную нагрузку и позволяет удерживать внимание пользователя на конкретном действии. Кроме того, понятная структура экранов облегчает дальнейшую поддержку кода и расширение функциональности.

1.2 Теоретическая база

Разработка клиентской части веб-приложения опирается на несколько базовых технологических понятий. HTML отвечает за структуру и семантику страницы, CSS формирует визуальное оформление, а JavaScript обеспечивает интерактивность. В рамках проекта именно JavaScript связывает пользовательские действия, локальное хранилище, внешний API и визуальное состояние интерфейса.

Для передачи данных между клиентом и сервером в современном вебе используется формат JSON. Он хорошо подходит для обмена объектами, так как легко сериализуется и парсится в JavaScript. В данном проекте JSON применяется при работе с локальными объектами пользователя, поездок, расходов и данных о погоде.

Важным аспектом теоретической базы является производительность. Для небольших интерфейсов использование тяжелых фреймворков не всегда оправдано, поскольку они увеличивают объем кода и усложняют поддержку. Для задачи курсовой работы достаточно нативного ES6+ подхода, при котором управление состоянием и DOM строится напрямую на возможностях браузера.

Таблица 1 - Сравнение подходов к клиентской разработке

Критерий	React / Vue / Angular	Чистый JavaScript	jQuery + плагины
Размер кода	Средний или большой	Минимальный	Средний
Сложность внедрения	Высокая	Низкая	Низкая
Порог входа	Высокий	Средний	Низкий
Производительность	Хорошая	Очень высокая	Средняя
Подходит для проекта	Избыточно	Оптимально	Устаревающий подход

Сравнение показывает, что для учебного проекта с небольшим количеством экранов наиболее оправдано использование чистого JavaScript.

Такой выбор позволяет сосредоточиться на архитектуре, логике и интерфейсе, а не на настройке фреймворка и дополнительных инструментов сборки.

1.3 Анализ конкурентных решений

На рынке существует множество решений, связанных с планированием путешествий. Крупные сервисы обычно предлагают поиск билетов, бронирование жилья, рекомендации по маршрутам и развитую карту. При этом их интерфейс зачастую перегружен и ориентирован на профессиональное использование или коммерческие сценарии.

Для учебной работы важнее не масштаб экосистемы, а качество пользовательского сценария. Пользователю Globe не требуется отдельный обучающий курс: ему достаточно открыть сайт, создать поездку и начать вводить полезную информацию. Это принципиально отличает проект от больших туристических платформ, где часть функций вообще может оказаться лишней.

С точки зрения удобства наиболее ценной оказалась идея единого пространства поездки. Вместо того чтобы хранить даты в календаре, расходы в таблицах, заметки в отдельном файле, а список вещей в мессенджере, пользователь получает одну карточку, где собраны все важные элементы. Такой подход особенно удобен для коротких поездок, командировок и учебных заданий.

Таблица 2 - Сравнение популярных решений для планирования поездок

Платформа	Регистрация	Скорость загрузки	Приватность	Гибкость сценария
Крупные туристические сервисы	Обязательна	Средняя	Ограниченная	Низкая
Карты и планировщики маршрутов	Часто обязательна	Средняя	Средняя	Средняя
Globe	Не обязательна для просмотра, необходима для персональных данных	Высокая	Высокая	Высокая

По результатам анализа можно сделать вывод, что разработанное решение занимает отдельную нишу: оно не конкурирует с глобальными сервисами по объему функций, но выигрывает за счет скорости, простоты и фокусировки на ключевых задачах путешественника.

1.4 Формирование требований к продукту

Требования к приложению делятся на функциональные и нефункциональные. Функциональные требования описывают, что именно должен уметь интерфейс. Нефункциональные требования определяют качество работы системы: удобство, скорость, безопасность и стабильность. Для курсовой работы важно зафиксировать оба типа требований, так как они напрямую влияют на архитектуру и выбор инструментов.

- Пользователь должен иметь возможность зарегистрироваться и войти в систему.
- После авторизации он должен видеть список собственных поездок.
- Пользователь должен создавать, открывать и удалять поездки.
- На странице поездки должны отображаться погода, бюджет, чек-лист и заметки.
- Интерфейс должен быть адаптивным и корректно работать на мобильных экранах.
- Вводимые данные должны проходить экранирование перед выводом в DOM.
- Тема оформления должна сохраняться между сеансами работы браузера.

Таблица 3 - Матрица рисков проекта

Риск	Вероятность	Влияние	Способ снижения
Потеря данных в localStorage	Средняя	Высокое	Пояснение пользователю о локальном хранении и резервные копии
XSS через пользовательский ввод	Средняя	Высокое	Экранирование текста при выводе
Сбой погодного API	Средняя	Среднее	Обработка ошибок и сообщение пользователю
Отсутствие	Высокая	Среднее	Перенос логики в полноценный сервер

синхронизации между устройствами			на следующем этапе
-------------------------------------	--	--	--------------------

Таким образом, на этапе анализа сформирована основа для последующего проектирования: определены границы системы, ключевые сценарии, ограничения и важные риски. Это позволяет перейти к архитектурным решениям без противоречий между задачей и реализацией.

ПРОЕКТИРОВАНИЕ И ОБОСНОВАНИЕ ВЫБОРА ТЕХНОЛОГИЙ

2.1 Концепция продукта

Globe задуман как легкое клиентское веб-приложение для персонального планирования путешествий. Центральная идея проекта состоит в том, чтобы пользователь мог быстро создать карточку поездки и затем последовательно наполнять ее полезной информацией. Такой подход уменьшает число действий, которые необходимо выполнить до начала работы с сервисом.

В основе концепции лежит принцип единого рабочего пространства. Все данные по поездке находятся в одном месте: даты, название, направление, бюджет, расходы, список вещей, заметки и погодные сведения. При этом пользователь не перегружается лишними экранами и формами, что особенно важно для мобильного сценария использования.

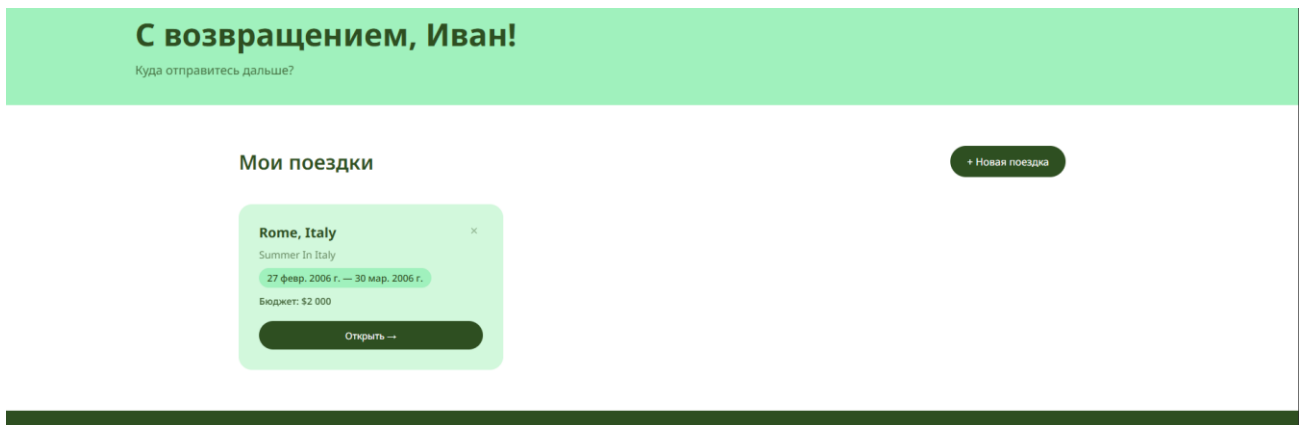


Рисунок 2 – Главный экран с созданными поездками

Структура экранов построена так, чтобы переход между ними был очевидным. Главная страница выполняет презентационную роль, дашборд служит точкой управления поездками, а страница поездки используется для ежедневной работы с конкретным маршрутом. Подобная композиция упрощает ориентацию в интерфейсе даже для пользователя, который впервые открыл сайт.

2.2 Целевая аудитория

Потенциальная аудитория проекта состоит из студентов, преподавателей, молодых специалистов, небольших команд и пользователей, которые регулярно планируют короткие поездки. Для этих групп особенно важны скорость, простота и отсутствие лишних барьеров. Необходима возможность быстро зафиксировать данные поездки и вернуться к ним позднее без сложного обучения.

Пользователь приложения не обязан обладать техническими навыками. Поэтому интерфейс должен объяснять сам себя: кнопки должны быть крупными, тексты — понятными, а состояния форм — прозрачными. Также важна поддержка адаптивной верстки, потому что большинство подобных задач решаются с телефона в дороге или вне рабочего места.

Для учебного проекта такая аудитория удобна еще и тем, что сценарии использования легко демонстрируются на защите: создание поездки, изменение бюджета, добавление списка вещей и просмотр погоды. Это делает продукт не абстрактной демонстрацией технологий, а конкретным инструментом с понятным назначением.

2.3 Обоснование выбора технологий

HTML5 используется как основа структуры страниц. Он позволяет выстроить семантическую разметку, корректно организовать формы, секции и заголовки. CSS3 отвечает за адаптивность, цветовые темы, визуальные состояния и компоновку блоков. JavaScript (ES6+) обеспечивает логику взаимодействия, обновление интерфейса, работу модальных окон и взаимодействие с хранилищем.

На стороне сервера используется Node.js с Express. Это решение минимально по сложности, но при этом достаточно для раздачи статических файлов и проксирования запросов к внешнему API. Сервер в проекте не

дублирует клиентскую логику, а служит вспомогательным элементом инфраструктуры.

Fetch API подходит для обмена данными в браузере, поскольку встроен в современные окружения и не требует дополнительных библиотек. Для хранения локального состояния выбран localStorage, так как для учебного проекта этого достаточно. Git используется для контроля версий и фиксации этапов разработки.

Таблица 4 - Обоснование выбора инструментов

Инструмент	Назначение	Причина выбора
HTML5	Разметка страниц	Семантичность и поддержка современных браузеров
CSS3	Оформление интерфейса	Адаптивность, Flexbox, Grid и поддержка тем
JavaScript ES6+	Интерактивность	Нативная работа с DOM и событиями
Node.js + Express	Локальный сервер	Простая настройка и удобный прокси для API
Fetch API	Получение данных	Базовый браузерный стандарт без лишних зависимостей
localStorage	Хранение данных	Подходит для персональных сведений в учебном проекте

Сравнительный анализ показывает, что выбранный стек закрывает поставленную задачу без избыточной инфраструктуры. Это особенно важно для учебного проекта, в котором необходимо показать качество интерфейса и архитектуры, а не объем подключенных зависимостей.

2.4 Анализ архитектуры и модульной структуры

Исходный код проекта разделен на независимые слои. Папка src содержит основную архитектуру приложения: страницы, блоки интерфейса, сервисы, утилиты и стили. Такое распределение упрощает сопровождение и позволяет точно вносить изменения, не затрагивая другие части системы. Визуальная часть в HTML-папке служит контейнером для подключения модулей.

Классы страниц отвечают за организацию экранов. Отдельные блоки реализуют повторно используемые элементы интерфейса, например навигацию, форму авторизации и переключатель темы. Сервисы инкапсулируют работу с

localStorage и внешним API, а утилиты содержат функции форматирования дат и безопасного вывода текста.

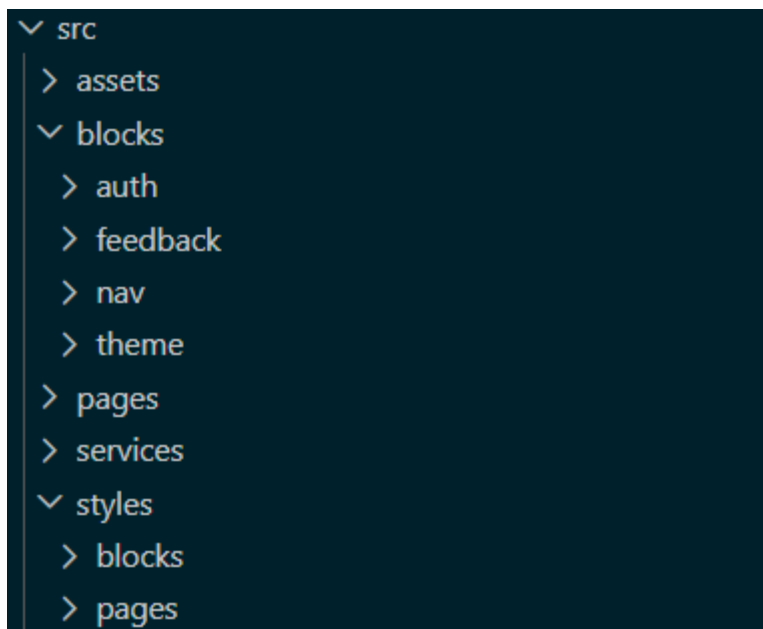


Рисунок 3 - Схема модульной архитектуры проекта

Подобная структура согласуется с принципами ООП и разделения ответственности. Каждый модуль выполняет одну задачу, а взаимодействие между ними происходит через четко определенные методы. Это уменьшает связанность кода и облегчает возможное расширение проекта в будущем.

2.5 Проектирование интерфейса

Пользовательский интерфейс строился по принципу минимализма и последовательности. Сначала пользователь видит рекламный и навигационный контент, затем получает возможность войти в систему, после чего переходит в рабочую зону с поездками. Такое расположение элементов снижает нагрузку на пользователя и делает путь к основной функции очевидным.

Для визуальной системы выбран спокойный современный стиль с контрастными кнопками и большим количеством свободного пространства. Это повышает читаемость и делает интерфейс более комфортным на разных устройствах. Отдельное внимание уделено адаптивности: блоки перестраиваются под узкие экраны без потери содержания.

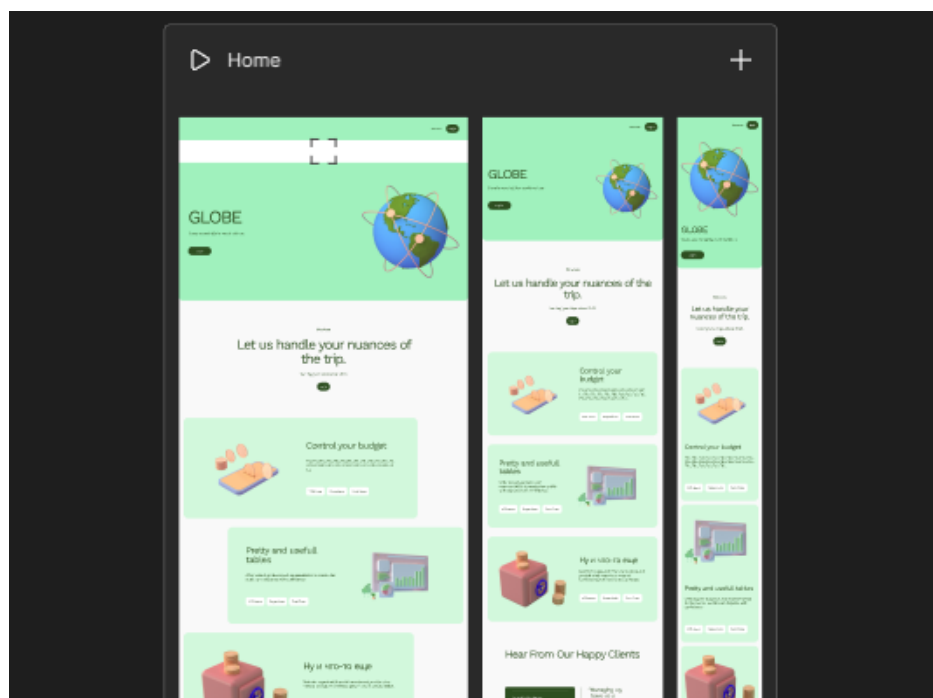


Рисунок 4 – Макет главной страницы в Figma

Отзывы наших клиентов

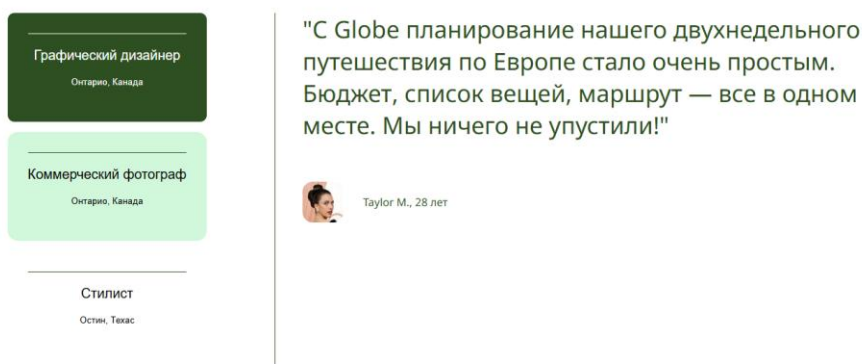


Рисунок 5 – Интерактивная секция с отзывами

Проектирование интерфейса учитывало также состояния пустых данных и ошибок. Пользователь должен понимать, что происходит, если поездок еще нет, если форма не заполнена или если внешний сервис недоступен. Эти состояния были заложены в дизайн наравне с основными экранами, а не добавлялись в проект как второстепенные элементы.

РЕАЛИЗАЦИЯ КЛИЕНТСКОЙ ЧАСТИ

3.1 Общая схема работы приложения

Точка входа приложения находится в файле `src/index.js`. После загрузки страницы скрипт определяет тип экрана по атрибуту `data-page` или по пути URL и подключает нужный класс страницы. Дополнительно сразу применяется сохраненная тема оформления, чтобы избежать визуального мерцания при переключении светлого и темного режима.

Такая организация позволяет использовать один модуль входа для нескольких HTML-страниц. Это упрощает структуру проекта, делает ее ближе к классической многовидовой веб-архитектуре и при этом не требует внешнего маршрутизатора. Основная логика разделяется по страницам, а повторяющиеся элементы выносятся в блоки.

Листинг 5 - Точка входа `src/index.js`

```
import { MainPage } from './pages/main-page.js';
import { DashboardPage } from './pages/dashboard-page.js';
import { TripPage } from './pages/trip-page.js';
import { ThemeToggle } from './blocks/theme/theme-toggle.js';

ThemeToggle.applySavedThemeEarly();

document.addEventListener('DOMContentLoaded', async () => {
  const theme = new ThemeToggle();
  theme.init();

  const page = document.body.dataset.page || getPageFromPath();

  if (page === 'main') {
    new MainPage().init();
    return;
  }

  if (page === 'dashboard') {
    new DashboardPage().init();
    return;
  }

  if (page === 'trip') {
    await new TripPage().init();
  }
});
```

Благодаря такому подходу каждый экран инициализируется только при необходимости. Это уменьшает число лишних обращений к DOM и делает

поведение приложения более предсказуемым. В результате код остается компактным, а логика не смешивается между экранами.

3.2 Реализация главной страницы

Главная страница выполняет презентационную функцию и одновременно служит точкой входа в систему. За нее отвечает класс `MainPage`, который сначала проверяет наличие авторизованного пользователя. Если пользователь уже вошел, он сразу перенаправляется на `dashboard.html`. Если нет, на странице активируются навигация, отзывы и окно авторизации.

Компонент навигации реализован классом `Nav`. Он создает кнопку-бургер для мобильного режима, скрывает и показывает элементы меню в зависимости от ширины экрана и обеспечивает плавный переход к нужным секциям страницы. Такой механизм делает навигацию удобной и на большом экране, и на телефоне.

Листинг 6 - Класс Nav

```
export class Nav {
  constructor({ navSelector, mobileWidth = 800 }) {
    this.nav = document.querySelector(navSelector);
    this.burgerButton = null;
    this.mobileWidth = mobileWidth;
    this.navButtons = [];
  }

  init() {
    if (!this.nav) return;

    this.navButtons = Array.from(this.nav.querySelectorAll('.nav__button'));
    this.burgerButton = this.createBurgerButton();
    this.nav.prepend(this.burgerButton);

    this.burgerButton.addEventListener('click', () => this.toggle());
    window.addEventListener('resize', () => this.applyResponsiveState());
    this.applyResponsiveState();
  }

  createBurgerButton() {
    const button = document.createElement('button');
    button.classList.add('burger-button', 'burger-button--open');
    button.innerHTML = '<svg xmlns="http://www.w3.org/2000/svg" width="22" height="22" viewBox="0 0 24 24" fill="none" stroke="currentColor" stroke-width="2.5" stroke-linecap="round"><line x1="3" y1="6" x2="21" y2="6"/><line x1="3" y1="12" x2="21" y2="12"/><line x1="3" y1="18" x2="21" y2="18"/></svg>';
    return button;
  }

  toggle() {
    if (!this.burgerButton) return;
  }
}
```


Листинг 6 - Класс Nav

```
const opened = this.nav.classList.toggle('nav--open');  
this.burgerButton.classList.toggle('burger-button--open', !opened);  
this.burgerButton.classList.toggle('burger-button--close', opened);  
this.applyResponsiveState();  
}  
}
```

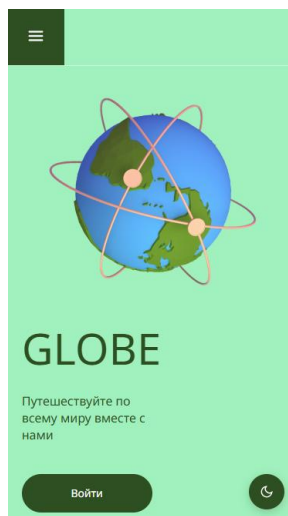


Рисунок 6 - Главная страница приложения с блоком навигации

Рисунок 7 - Модальное окно авторизации и регистрации

Форма авторизации и регистрации реализована в модуле AuthModal. Пользовательские данные сохраняются в localStorage под отдельным пространством имен. Для учебного проекта этого достаточно, так как основная задача состоит в демонстрации интерфейса и логики работы с локальным состоянием, а не в организации промышленной серверной безопасности.

Листинг 7 - Класс AuthModal

```
export class AuthModal {
  constructor() {
    this.storage = new StorageService('globe');
    this.authModal = document.getElementById('authModal');
  }

  init() {
    if (!this.authModal) return;

    this.bindOpenButtons();
    this.bindCloseActions();
    this.bindTabs();
    this.bindForms();
  }

  bindOpenButtons() {
    document.querySelectorAll('.login-button').forEach(btn => {
      btn.addEventListener('click', () => this.open());
    });
  }

  handleRegister() {
    const name = document.getElementById('regName').value.trim();
    const email = document.getElementById('regEmail').value.trim().toLowerCase();
    const password = document.getElementById('regPassword').value;
    const registerError = document.getElementById('registerError');

    const users = this.storage.getUsers();
    const existing = users.find((u) => u.email === email);

    if (existing) {
      if (registerError) {
        registerError.textContent = 'Эта почта уже зарегистрирована';
      }
      return;
    }

    users.push({ name, email, password });
    this.storage.setUsers(users);
    this.storage.setCurrentUser({ name, email });
    window.location.href = 'dashboard.html';
  }
}
```

3.3 Реализация дашборда пользователя

Страница `dashboard` является центром управления поездками. Здесь пользователь видит приветствие, список собственных поездок и кнопку создания новой записи. Если поездок пока нет, отображается пустое состояние, которое подсказывает пользователю, что можно начать планирование с первого маршрута.

Класс `DashboardPage` получает текущего пользователя из сервиса хранения, выводит его имя в шапке и формирует набор карточек поездок.

Каждая карточка содержит название, направление, даты, бюджет и кнопку перехода к подробному экрану. Для удаления предусмотрено отдельное подтверждение, которое исключает случайное действие.

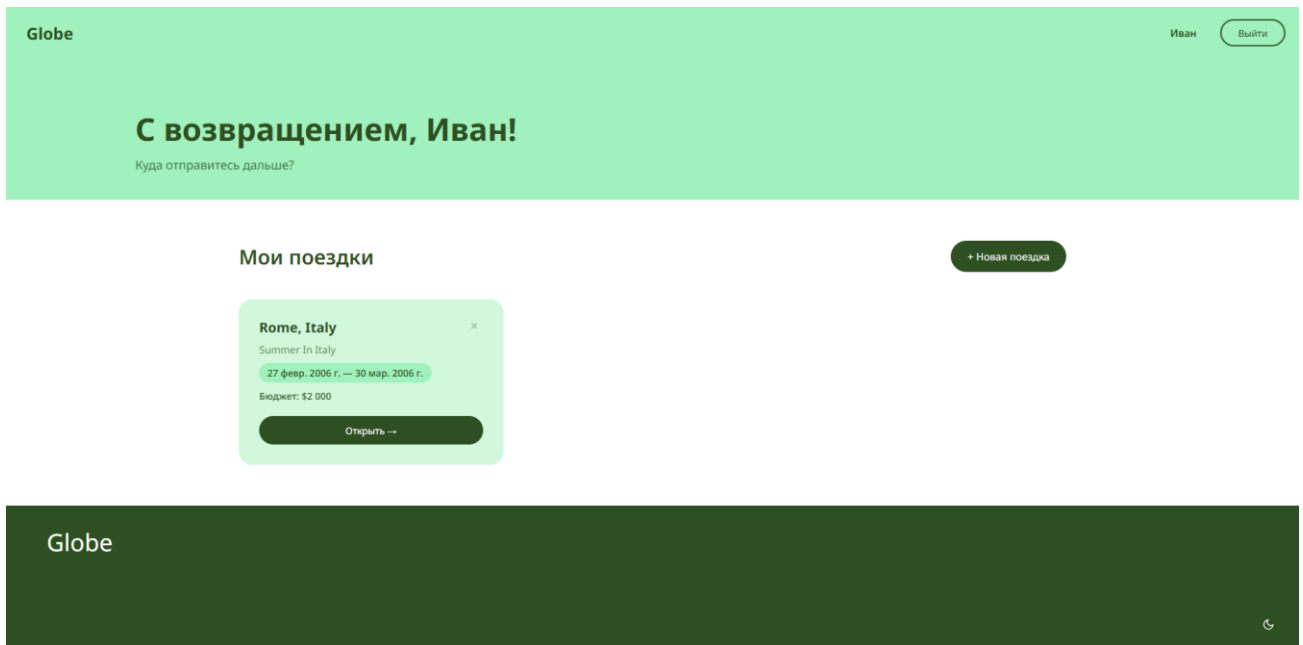


Рисунок 8 - Панель пользователя со списком поездок

Листинг 8 - Класс *DashboardPage*

```
export class DashboardPage {
  constructor() {
    this.storage = new StorageService('globe');
    this.currentUser = null;
    this.tripToDelete = null;
  }

  init() {
    this.currentUser = this.storage.getCurrentUser();
    if (!this.currentUser) {
      window.location.href = 'index.html';
      return;
    }

    document.getElementById('navUsername').textContent = this.currentUser.name;
    document.getElementById('greeting').textContent = 'С возвращением, ' +
this.currentUser.name + '!';

    document.getElementById('logoutBtn').addEventListener('click', () => {
      this.storage.clearCurrentUser();
      window.location.href = 'index.html';
    });

    this.bindModals();
    this.bindTripForm();
    this.bindDeleteModal();
  }
}
```

Листинг 8 - Класс *DashboardPage*

```
        this.renderTrips();
    }

    renderTrips() {
        const trips = this.getTrips();
        const grid = document.getElementById('tripsGrid');
        const empty = document.getElementById('tripsEmpty');

        grid.innerHTML = '';

        if (trips.length === 0) {
            empty.classList.add('trips-empty--visible');
            return;
        }
    }
}
```

При создании новой поездки проверяются обязательные поля и даты. Если дата окончания раньше даты начала, форма не сохраняется, а пользователь получает понятное сообщение об ошибке. Такой контроль важен не только для корректности данных, но и для демонстрации формы в учебной среде.

3.4 Реализация страницы поездки

Страница `trip.html` объединяет наиболее функционально насыщенные элементы приложения. Класс `TripPage` получает `id` поездки из адресной строки, находит объект в `localStorage`, отображает шапку поездки и подгружает дополнительные блоки: погоду, бюджет, чек-лист и заметки. Если поездка не найдена, пользователь возвращается в дашборд.

Погодный блок использует внешний API через локальный сервер. Сначала сервер получает координаты города, затем запрашивает прогноз у Open-Meteo и возвращает клиенту упрощенный объект с температурой, ветром, кодом погоды и названием населенного пункта. Это позволяет скрыть техническую сложность запроса от интерфейса.

Рисунок 9 - Страница поездки с блоком прогноза погоды

Бюджет

Общий бюджет:

\$2 000

Потрачено:

\$250

Осталось: \$1 750

Описание (например, отель)

Сумма (\$)

+ Добавить

Отель

\$250

×

Рисунок 10 – Блок бюджета и расходов

Список вещей

Добавить пункт (например, паспорт)

+ Добавить

☒ Паспорт

×

☐ Щетка

×

1 / 2 вещей собрано

Заметки

Записывайте все о поездке — адреса отелей, советы, идеи...

Рисунок 11 – Чек-лист и заметки пользователя

Листинг 9 - Класс *TripPage*

```
export class TripPage {
  constructor() {
    this.storage = new StorageService('globe');
    this.weather = new WeatherService();
    this.currentUser = null;
    this.tripId = null;
    this.saveTimeout = null;
  }

  async init() {
    this.currentUser = this.storage.getCurrentUser();
    if (!this.currentUser) {
      window.location.href = 'index.html';
      return;
    }

    document.getElementById('navUsername').textContent = this.currentUser.name;
    document.getElementById('logoutBtn').addEventListener('click', () => {
      this.storage.clearCurrentUser();
      window.location.href = 'index.html';
    });

    const params = new URLSearchParams(window.location.search);
    this.tripId = params.get('id');

    if (!this.tripId) {
      window.location.href = 'dashboard.html';
      return;
    }

    const trip = this.getTrip();
    if (!trip) {
      window.location.href = 'dashboard.html';
      return;
    }

    this.renderHero(trip);
    this.ensureTripCollections(trip);
    this.bindBudgetEvents();
    this.bindChecklistEvents();
    this.bindNotesEvents();

    this.renderBudget();
    this.renderChecklist();
    this.renderNotes();
    await this.loadWeather();
  }
}
```

Бюджет реализован как динамический список расходов. Пользователь вводит название траты и сумму, после чего система пересчитывает общий расход и остаток. Если значение превышает установленный бюджет, индикатор меняет состояние, а текст подсказывает о перерасходе. Это делает интерфейс не только информативным, но и полезным в повседневной практике.

Листинг 10 - Сервис погоды WeatherService

```
export class WeatherService {
  async getCurrentWeather(city) {
    const response = await fetch(`/api/weather?city=${encodeURIComponent(city)}`);
    if (!response.ok) {
      throw new Error('weather_request_failed');
    }
    return response.json();
  }

  getDescription(code) {
    const descriptions = {
      0: 'Ясно',
      1: 'Преимущественно ясно',
      2: 'Переменная облачность',
      3: 'Пасмурно',
      45: 'Туман',
      48: 'Туман',
      51: 'Небольшой дождь',
      53: 'Умеренный дождь',
      55: 'Сильный дождь',
      80: 'Ливневый дождь',
      81: 'Сильный ливень',
      99: 'Гроза'
    };

    return descriptions[code] || 'Погода';
  }
}
```

3.5 Реализация хранения данных

Хранение данных вынесено в отдельный сервис StorageService. Он инкапсулирует работу с localStorage и предоставляет методы для получения текущего пользователя, списка пользователей и списка поездок. Такой подход уменьшает количество прямых обращений к хранилищу в классах страниц и упрощает потенциальный переход к другой системе хранения.

Локальное хранилище удобно для учебной версии приложения, поскольку не требует настройки базы данных и позволяет быстро показать результат в браузере. При этом стоит помнить, что localStorage не предназначен для серьезных распределенных сценариев. Поэтому в тексте отчета это решение рассматривается именно как этап прототипирования.

Листинг 11 - Сервис хранения StorageService

```
export class StorageService {
  constructor(namespace = 'globe') {
    this.namespace = namespace;
  }

  getCurrentUser() {
```

Листинг 11 - Сервис хранения *StorageService*

```
        return JSON.parse(localStorage.getItem(`${this.namespace}_current_user`));
    }

    setCurrentUser(user) {
        localStorage.setItem(`${this.namespace}_current_user`, JSON.stringify(user));
    }

    clearCurrentUser() {
        localStorage.removeItem(`${this.namespace}_current_user`);
    }

    getUsers() {
        return JSON.parse(localStorage.getItem(`${this.namespace}_users`)) || [];
    }

    setUsers(users) {
        localStorage.setItem(`${this.namespace}_users`, JSON.stringify(users));
    }

    getTrips(email) {
        return JSON.parse(localStorage.getItem(`${this.namespace}_trips_${email}`)) ||
        [];
    }
}
```

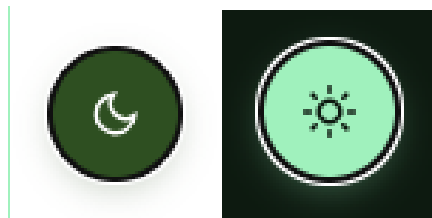


Рисунок 12 - Диалог сохранения темы оформления и переключатель темы

Переключение темы реализовано через отдельный компонент ThemeToggle. Он запоминает выбор пользователя в localStorage и применяет его при следующем открытии страницы. Такое решение не только повышает удобство, но и демонстрирует работу с persistent state на стороне клиента.

Листинг 12 - Компонент ThemeToggle

```
export class ThemeToggle {
  constructor() {
    this.button = document.getElementById('themeToggle');
    this.moonSVG = '<svg xmlns="http://www.w3.org/2000/svg" width="18" height="18" viewBox="0 0 24 24" fill="none" stroke="currentColor" stroke-width="2" stroke-linecap="round"><path d="M21 12.79A9 9 0 1 1 11.21 3 7 7 0 0 0 21 12.79z"/></svg>';
    this.sunSVG = '<svg xmlns="http://www.w3.org/2000/svg" width="18" height="18" viewBox="0 0 24 24" fill="none" stroke="currentColor" stroke-width="2" stroke-linecap="round"><circle cx="12" cy="12" r="5"/></svg>';
  }

  static applySavedThemeEarly() {
    const saved = localStorage.getItem('globe_theme') || 'light';
    document.documentElement.setAttribute('data-theme', saved);
  }

  init() {
    if (!this.button) return;
    this.updateButton();
    this.button.addEventListener('click', () => {
      const current = document.documentElement.getAttribute('data-theme');
      const next = current === 'dark' ? 'light' : 'dark';
      document.documentElement.setAttribute('data-theme', next);
      localStorage.setItem('globe_theme', next);
      this.updateButton();
    });
  }
}
```

3.6 Работа с утилитами и безопасным выводом

Для вспомогательных операций в проекте предусмотрены небольшие утилиты. Функция `formatDateRu` преобразует дату в читабельный формат для пользователя, а `tripDaysCount` позволяет определить количество дней в поездке. Функция `escapeHTML` защищает от XSS, так как преобразует пользовательский ввод в безопасный текст перед отображением на странице.

Листинг 13 - Утилиты работы с датой и DOM

```
export function formatDateRu(dateStr) {
  if (!dateStr) return '';
  const d = new Date(`${dateStr}T00:00:00`);
  return d.toLocaleDateString('ru-RU', {
    month: 'short',
    day: 'numeric',
    year: 'numeric'
  });
}

export function tripDaysCount(startDate, endDate) {
  if (!startDate || !endDate) return 0;
  const start = new Date(`${startDate}T00:00:00`);
  const end = new Date(`${endDate}T00:00:00`);
  const days = Math.round((end - start) / (1000 * 60 * 60 * 24)) + 1;
  return days > 0 ? days : 0;
}
```

Листинг 14 - Функция безопасного вывода текста

```
export function escapeHTML(str) {  
    const div = document.createElement('div');  
    div.appendChild(document.createTextNode(String(str ?? '')));  
    return div.innerHTML;  
}  
  
export function byId(id) {  
    return document.getElementById(id);  
}
```

ТЕСТИРОВАНИЕ, ДОКУМЕНТИРОВАНИЕ И ПУБЛИКАЦИЯ

4.1 Подход к тестированию

Проверка работоспособности проекта выполнялась вручную по основным пользовательским сценариям. Для учебного фронтенд-проекта такой формат тестирования оправдан, поскольку основная задача состоит в проверке корректности интерфейса, логики событий и взаимодействия с хранилищем. Кроме того, при небольшом объеме кода ручная проверка дает наглядный результат и позволяет быстро исправлять выявленные несоответствия.

Тестирование охватывало вход пользователя, переход на дашборд, создание поездки, открытие подробной карточки, работу с погодным блоком, добавление расходов, ведение чек-листа, сохранение заметок и переключение темы. Отдельно проверялись пустые состояния и реакции на некорректный ввод.

Таблица 5 - Результаты проверки основных сценариев

Тест-кейс	Ожидаемый результат	Фактический результат	Статус
Вход зарегистрированного пользователя	Открывается дашборд	Открывается дашборд	Пройден
Создание новой поездки	Появляется карточка поездки	Появляется карточка поездки	Пройден
Открытие страницы поездки	Загружаются данные по id	Загружаются данные по id	Пройден
Получение погоды	Отображается текущая погода	Отображается текущая погода	Пройден
Добавление расходов	Обновляется сумма бюджета	Обновляется сумма бюджета	Пройден
Переключение темы	Тема меняется и сохраняется	Тема меняется и сохраняется	Пройден

По итогам тестирования можно сделать вывод, что базовые сценарии функционируют корректно. Пользовательские данные сохраняются между обновлениями страницы, а интерфейс устойчиво реагирует на отсутствие авторизации и ошибки во внешнем сервисе.

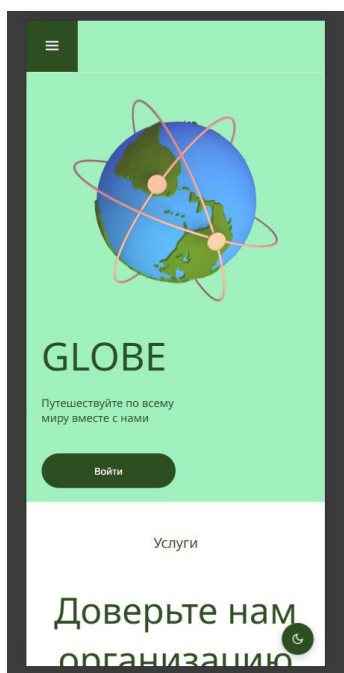


Рисунок 13 - Проверка работы интерфейса на мобильном экране

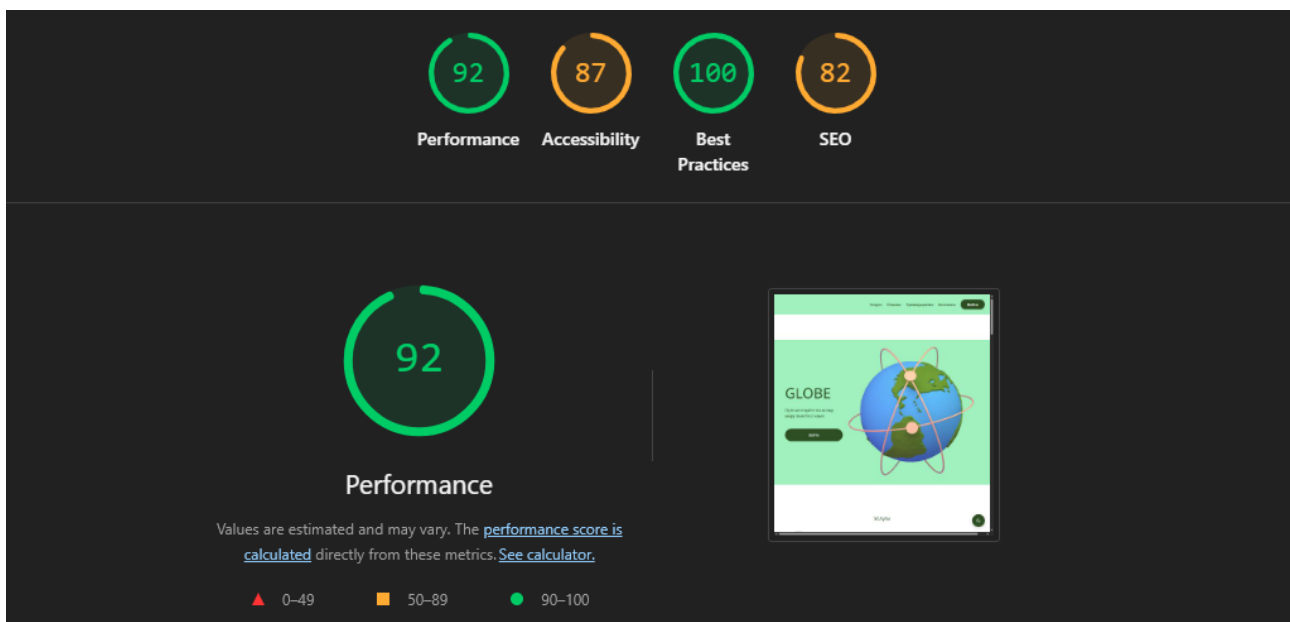


Рисунок 14 - Результат работы Google Lighthouse

Дополнительно оценивалась адаптивность верстки. На мобильных устройствах сохраняется читаемость текстов, кнопки остаются достаточно крупными для нажатия пальцем, а блоки перестраиваются без потери содержания. Это важно для реального пользовательского сценария, так как путешественник часто работает с приложением именно со смартфона.

4.2 Документирование проекта

Для проекта подготовлена сопроводительная документация. В нее входят описание структуры каталогов, краткие инструкции по запуску, а также пояснения по назначению основных модулей. Такая документация облегчает передачу проекта другому разработчику и ускоряет последующее развитие.

Файл ARCHITECTURE.md содержит описание модульной структуры src и поясняет принципы разделения страниц, блоков, сервисов и утилит. Это особенно полезно в учебном проекте, где требуется демонстрировать не только конечный интерфейс, но и подход к организации кода.

Листинг 15 - Описание архитектурного слоя src

```
# src architecture

This directory contains the modular architecture version of the project for coursework requirements.

## Structure
- `blocks/` - BEM-based UI modules
- `pages/` - page orchestrators
- `services/` - APIs and storage access
- `utils/` - helper utilities
- `styles/` - shared style primitives
- `assets/` - media placeholders

## Migration strategy
1. Keep `frontend/static/js` as stable production code.
2. Move one page at a time to `src/pages/*`.
3. Connect bundler later if required by next assignment stage.
```

Таким образом, документация проекта не ограничивается отдельным README. Архитектурное описание помогает объяснить логику построения приложения и делает отчет более понятным для проверки преподавателем.

4.3 Публикация и запуск

Локальный запуск выполняется через сервер server.js. Express раздает статические файлы из папки frontend и подключает каталог src для модулей и стилей. Дополнительно сервер предоставляет маршрут /api/weather, который скрывает обращение к внешнему погодному сервису и возвращает клиенту уже обработанный JSON-ответ.

Листинг 16 - Серверная часть server.js

```
const express = require('express');
const path = require('path');

const app = express();
const PORT = process.env.PORT || 5500;

app.use((req, res, next) => {
  res.setHeader('Access-Control-Allow-Origin', '*');
  res.setHeader('Access-Control-Allow-Methods', 'GET, OPTIONS');
  res.setHeader('Access-Control-Allow-Headers', 'Content-Type');
  if (req.method === 'OPTIONS') {
    return res.sendStatus(204);
  }
  next();
});

app.use(express.static(path.join(__dirname, 'frontend')));
app.use('/src', express.static(path.join(__dirname, 'src')));

app.get('/api/weather', async (req, res) => {
  const city = String(req.query.city || '').trim();
  if (!city) {
```

Листинг 16 - Серверная часть server.js

```
    return res.status(400).json({ error: 'City is required' });
  }

  try {
    const geocodeUrl = `https://geocoding-api.open-
meteo.com/v1/search?name=${encodeURIComponent(city)}&count=1&language=en&format=json`;
    const geocodeResponse = await fetch(geocodeUrl);
    const geocodeJson = await geocodeResponse.json();
    const location = geocodeJson.results[0];
    const weatherUrl = `https://api.open-
meteo.com/v1/forecast?latitude=${location.latitude}&longitude=${location.longitude}&curr
ent=temperature_2m,wind_speed_10m,weather_code`;
    const weatherResponse = await fetch(weatherUrl);
    const weatherJson = await weatherResponse.json();

    return res.json({
      city: location.name,
      country: location.country || '',
      temperature: weatherJson.current?.temperature_2m,
      windSpeed: weatherJson.current?.wind_speed_10m,
      weatherCode: weatherJson.current?.weather_code
    });
  } catch (error) {
    return res.status(500).json({ error: 'Internal server error' });
  }
});
```

Размещение проекта на статическом хостинге возможно для фронтенд-части, но локальный сервер остается полезным для демонстрации интеграции с погодным API. В учебной версии это достаточно для показа полного цикла работы клиента с внешним источником данных.

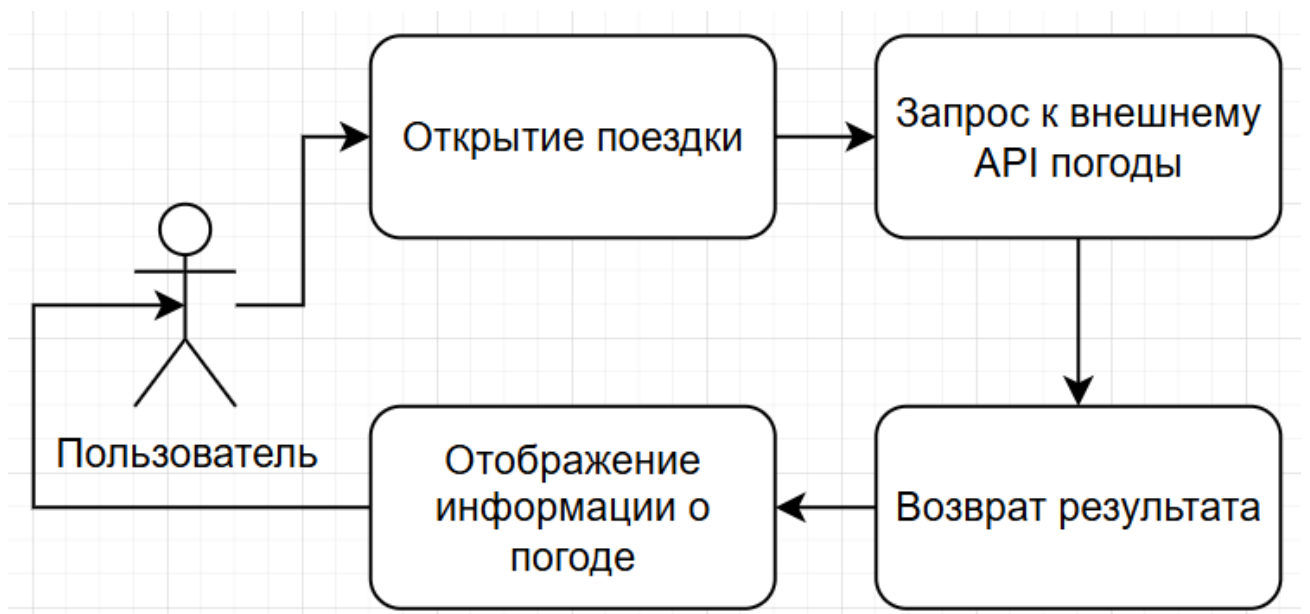


Рисунок 15 - Схема взаимодействия клиента с внешним погодным API

ЗАКЛЮЧЕНИЕ

В ходе курсовой работы была разработана клиентская часть веб-приложения Globe для планирования путешествий. Реализованы главная страница, панель пользователя и подробная страница поездки, а также механизмы авторизации, локального хранения данных, ведения бюджета, чек-листа, заметок и получения погодной информации через внешний API.

Поставленная цель достигнута. Проект демонстрирует применение HTML, CSS, JavaScript, Express, localStorage и Fetch API в рамках единой модульной архитектуры. В ходе выполнения работы были также закреплены принципы БЭМ-структурирования, ООП-подхода и адаптивной верстки.

Практическая ценность решения заключается в том, что приложение можно использовать как основу для дальнейшего развития. Перспективными направлениями являются перенос данных в серверную базу, добавление облачной синхронизации, совместное редактирование поездок, более надежная система авторизации и расширенная аналитика расходов.

Сформированный отчет содержит анализ предметной области, описание архитектуры, сведения о реализации, результаты тестирования и список использованных источников. Таким образом, курсовая работа выполнена в полном объеме и может быть представлена к защите как законченный учебный проект.

СПИСОК ИСПОЛЬЗУЕМЫХ ИСТОЧНИКОВ

1. MDN Web Docs. HTML. URL: <https://developer.mozilla.org/ru/docs/Web/HTML>
2. MDN Web Docs. CSS. URL: <https://developer.mozilla.org/ru/docs/Web/CSS>
3. MDN Web Docs. JavaScript. URL: <https://developer.mozilla.org/ru/docs/Web/JavaScript>
4. MDN Web Docs. Fetch API. URL: https://developer.mozilla.org/ru/docs/Web/API/Fetch_API
5. MDN Web Docs. Window.localStorage. URL: <https://developer.mozilla.org/ru/docs/Web/API/Window/localStorage>
6. Express documentation. URL: <https://expressjs.com/>
7. Open-Meteo API documentation. URL: <https://open-meteo.com/>
8. Node.js documentation. URL: <https://nodejs.org/ru/docs>
9. БЭМ. Методология. URL: <https://ru.bem.info/>
10. Git documentation. URL: <https://git-scm.com/doc>
11. web.dev. Responsive design. URL: <https://web.dev/responsive-web-design-basics/>
12. MDN Web Docs. Form validation. URL: https://developer.mozilla.org/ru/docs/Learn_web_development/Extensions/Forms/Form_validation

ПРИЛОЖЕНИЕ А

Листинг А.1 - Дерево каталогов проекта

```
frontend/  
├── dashboard.html  
├── index.html  
├── trip.html  
└── static/  
src/  
├── index.js  
├── pages/  
├── blocks/  
├── services/  
├── utils/  
├── styles/  
└── assets/  
server.js  
package.json
```

Папка frontend содержит HTML-страницы и статические изображения. Папка src содержит основные модули приложения, стили, блоки и сервисы. Такое разделение упрощает навигацию по проекту и поддерживает модульный подход к разработке.

ПРИЛОЖЕНИЕ Б

Листинг В.1 - Класс MainPage

```
export class MainPage {
  constructor() {
    this.storage = new StorageService('globe');
  }

  init() {
    const currentUser = this.storage.getCurrentUser();
    if (currentUser) {
      window.location.href = 'dashboard.html';
      return;
    }

    const nav = new Nav({ navSelector: '.nav' });
    nav.init();
    nav.bindAnchors();

    const feedback = new FeedbackTabs();
    feedback.init();

    const authModal = new AuthModal();
    authModal.init();
  }
}
```

Листинг В.2 - Класс FeedbackTabs

```
export class FeedbackTabs {
  constructor(buttonSelector = '.feedback-table-element', reviewSelector =
    '.feedback-review') {
    this.buttons = document.querySelectorAll(buttonSelector);
    this.reviews = document.querySelectorAll(reviewSelector);
  }

  init() {
    this.buttons.forEach((button) => {
      button.addEventListener('click', () => this.activate(button.id));
    });
  }

  activate(id) {
    this.buttons.forEach((button) => {
      button.classList.toggle('feedback-table-element--active', button.id ===
id);
    });

    this.reviews.forEach((review) => {
      review.classList.toggle('feedback-review--active', review.id === id);
    });
  }
}
```

Листинг В.3 - Фрагмент storage-service.js

```
export class StorageService {
  constructor(namespace = 'globe') {
    this.namespace = namespace;
  }

  getCurrentUser() {
    return JSON.parse(localStorage.getItem(`${this.namespace}_current_user`));
  }

  setCurrentUser(user) {
    localStorage.setItem(`${this.namespace}_current_user`, JSON.stringify(user));
  }

  clearCurrentUser() {
    localStorage.removeItem(`${this.namespace}_current_user`);
  }
}
```

Листинг В.4 - Фрагмент weather-service.js

```
export class WeatherService {
  async getCurrentWeather(city) {
    const response = await fetch(`/api/weather?city=${encodeURIComponent(city)}`);
    if (!response.ok) {
      throw new Error('weather_request_failed');
    }
    return response.json();
  }
}
```

ПРИЛОЖЕНИЕ В

Листинг Г.1 - Маршрут /api/weather

```
app.get('/api/weather', async (req, res) => {
  const city = String(req.query.city || '').trim();

  if (!city) {
    return res.status(400).json({ error: 'City is required' });
  }

  try {
    const geocodeUrl = `https://geocoding-api.open-
meteo.com/v1/search?name=${encodeURIComponent(city)}&count=1&language=en&format=json`;
    const geocodeResponse = await fetch(geocodeUrl);
    const geocodeJson = await geocodeResponse.json();
    const location = geocodeJson.results[0];
    const weatherUrl = `https://api.open-
meteo.com/v1/forecast?latitude=${location.latitude}&longitude=${location.longitude}&curr
ent=temperature_2m,wind_speed_10m,weather_code`;
    const weatherResponse = await fetch(weatherUrl);
    const weatherJson = await weatherResponse.json();

    return res.json({
      city: location.name,
      country: location.country || '',
      temperature: weatherJson.current?.temperature_2m,
      windSpeed: weatherJson.current?.wind_speed_10m,
      weatherCode: weatherJson.current?.weather_code
    });
  } catch (error) {
    return res.status(500).json({ error: 'Internal server error' });
  }
});
```

Листинг Г.2 - Команда запуска проекта

```
{
  "scripts": {
    "dev": "node server.js",
    "start": "node server.js"
  },
  "dependencies": {
    "express": "^5.2.1"
  }
}
```